

AI-Powered Smart City Insights Platform

Final Report — UPC CCBDA Team 11

Jia Lyu Jakub Jozef Dusza José Ricardo Arias Perez
Mark Welf Atzberger

May 29, 2026

Contents

1	Introduction: What the Project Finally Does	3
1.1	What a User Can Ask	3
1.2	User-Facing Surfaces	3
1.3	MCP Tools Summary	4
2	Scope Changes from the First Draft	4
2.1	Features Added Beyond the Proposal	4
2.2	Features Dropped or Simplified	5
3	Architecture and Code Description	6
3.1	DATASOURCES	6
3.2	Layered Architecture	6
3.3	Data Ingestion Lambdas	7
3.4	DynamoDB Schema Design	8
3.5	MCP Server (<code>mcp_server.py</code>)	9
3.6	Demo Web Application (<code>demo/app.py</code>)	9
3.7	Publicly Deployed Bedrock Chat Application	10
3.8	AWS Resources	10
4	Twelve-Factor Methodology	11
5	Development Methodology	13
5.1	Team Organisation and Ownership Model	13
5.2	Communication and Shared Documentation	14
5.3	Working Environments	14
6	Problems Encountered and Solutions	14
6.1	Dead Data Sources	14
6.2	Lambda Function URL Account-Level Block	15
6.3	IAM 10 Managed Policy Limit	15
6.4	UV Index Nighttime Edge Case	15
6.5	innerHTML Destroys Event Listeners	15

6.6	Pollen History Bootstrap Problem	15
6.7	Bicing BSM API Block	16
7	Services and Resources Used	16
7.1	AWS Lambda	16
7.2	Amazon DynamoDB	16
7.3	Amazon EventBridge Scheduler	16
7.4	Amazon API Gateway (HTTP API)	16
7.5	AWS Bedrock	17
7.6	Amazon S3 + CloudFront	17
7.7	AWS Bedrock AgentCore Gateway	17
7.8	MCP (Model Context Protocol)	17
7.9	Open Data Sources	18
8	Hours Invested	18
8.1	Analysis of the Deviation	19
8.2	How the Deviation Could Have Been Reduced	19
9	Conclusion	20

1 Introduction: What the Project Finally Does

The Barcelona Smart City Insights Platform is a live city data infrastructure built entirely on AWS that makes six types of real-time urban data accessible to any compatible AI assistant through a standardised protocol interface. The system ingests data continuously from Barcelona’s public open APIs, stores it in Amazon DynamoDB, and exposes it via an MCP (Model Context Protocol) server deployed on AWS Lambda and API Gateway.

The core value proposition is straightforward: Barcelona publishes excellent open data, but that data is practically inaccessible in conversational AI tools that citizens already use every day. By building an MCP server on top of AWS infrastructure, any AI assistant that supports MCP gains the ability to answer real questions about the real state of the city right now — bikes, transport, pollution, weather, UV index, and pollen — without requiring a custom app or any change in user behaviour.

1.1 What a User Can Ask

A user who connects an AI Agent (e.g. Claude, ChatGPT) to the MCP endpoint can ask questions such as:

- *“Are there Bicing bikes available near Sagrada Família right now?”* — returns live station counts, mechanical vs. electric split, distances from the requested location, and online/offline status.
- *“Is today a good day to go for a run in Eixample?”* — the AI agent calls weather, UV, air quality, and pollen in a single turn and synthesises a composite actionable answer.
- *“What was the worst hour for NO₂ pollution in Gràcia this week?”* — queries the hourly history DynamoDB table and identifies the peak reading with its timestamp.
- *“How do I get from UPC Campus Nord to Barceloneta by public transit?”* — returns real journey options with legs, line names, and total duration via the Transitous open router.
- *“I am allergic to grass pollen — how bad is today and when is the worst window?”* — returns grains/m³ with level classification and the peak forecast hour.

1.2 User-Facing Surfaces

The platform has three access points:

1. **MCP server endpoint** (<https://911xtl8mm3.execute-api.eu-west-1.amazonaws.com/mcp>): any MCP-compatible AI client can add this URL as a custom connector and immediately use all 11 tools in natural conversation.
2. **Public Bedrock chatbot** (<https://dhioxm566jvi3.cloudfront.net>): a self-contained web page that runs its own agentic loop via AWS Bedrock and the Claude Haiku model. The user opens the page in a browser, types a question, and the backend orchestrates tool calls to the MCP server before returning a synthesised answer with a collapsible activity log of every tool invoked.

3. **Local demo web app** (`demo/app.py`): a FastAPI application with a Leaflet map, an AI chatbot backed by Bedrock, and toggleable city data overlays (air quality stations, metro stops, bus stops). Used for course demonstrations.

1.3 MCP Tools Summary

The platform exposes 11 MCP tools across six data domains. All data is free and requires no API keys from the user side. Table 1 summarises the tools.

Table 1: MCP tools exposed by the Barcelona Smart City platform

Tool	Domain	Data Source	Update cadence
<code>get_bicing</code>	Mobility	Bicing GBFS (citybik.es)	Every 5 min
<code>get_bicing_history</code>	Mobility	DynamoDB (BicingStations)	Every 5 min
<code>get_transit_nearby</code>	Mobility	DynamoDB (TransitStops)	Static GTFS
<code>get_transit_route</code>	Mobility	Transitous open router	Per request
<code>get_air_quality</code>	Air Quality	DynamoDB (AirQualityReadings)	Hourly
<code>get_air_quality_history</code>	Air Quality	DynamoDB (AirQualityReadings)	Hourly
<code>get_weather</code>	Weather	DynamoDB (WeatherData)	Hourly
<code>get_uv_index</code>	UV	currentuvindex.com (CAMS)	Per request
<code>get_uv_history</code>	UV	DynamoDB (UVData)	Hourly
<code>get_pollen</code>	Pollen	Open-Meteo CAMS	Per request
<code>get_pollen_history</code>	Pollen	DynamoDB (PollenData)	Hourly

2 Scope Changes from the First Draft

2.1 Features Added Beyond the Proposal

UV index vertical. The project proposal did not mention UV data. During vertical research, currentuvindex.com was identified as a free, reliable source providing hourly UV index values from the Copernicus [REFERENCE](#) CAMS model. A `uv_ingest` Lambda was written to store readings in a `UVData` DynamoDB table every hour. The `get_uv_index` MCP tool includes the Diffey formula for burn time estimation (minutes to sunburn for unprotected fair skin at a given UV value), WHO risk categories (low / moderate / high / very high / extreme), today’s hourly forecast, and the 5-day peak. A matching `get_uv_history` tool was added for trend analysis.

Pollen vertical. The proposal also did not include pollen data. The Open-Meteo CAMS air quality model provides hourly pollen concentration forecasts for six allergen species (grass, birch, olive, mugwort, ragweed, alder) at no cost, at any coordinate. A `pollen_ingest` Lambda writes one record per species per hour to `PollenData`. The `get_pollen` tool returns current concentration, level badge, dominant allergen, and 96-hour forecast peak per species. A `get_pollen_history` tool supports historical trend queries.

Historical query tools for all verticals. The first draft described only real-time lookups. The final system adds four history tools — `get_bicing_history`, `get_air_quality_history`,

`get_uv_history`, `get_pollen_history` — each backed by a DynamoDB table that retains 30 days of readings. This enables trend queries and peak identification that the proposal did not anticipate, and they turned out to be among the most compelling demo prompts.

Extended air quality station coverage. The proposal mentioned four monitoring stations; the final implementation covers nine XVPCA stations across Barcelona’s districts: Poble Nou, Sants, Eixample, Gràcia, Ciutatella, Vall Hebron, Palau Reial, Observatori Fabra, and Navas. The nearest-station selection uses the Haversine formula over the full set.

Publicly deployed Bedrock chat application. The Bedrock chatbot is now publicly accessible at a CloudFront URL. It uses a Lambda-based agentic loop, S3-hosted static frontend, and CloudFront CDN.

AWS Bedrock AgentCore Gateway. During development, an AgentCore Gateway was configured to auto-discover and proxy all 11 MCP tools. Although the final chat Lambda calls the MCP server directly, the gateway is deployed and available at a managed endpoint for future use with Bedrock Agents or the Strands SDK.

2.2 Features Dropped or Simplified

Noise vertical. The proposal assigned the noise vertical (Sentilo acoustic sensor network) to José. After building and deploying the Lambda, it was discovered that the Sentilo API (`connecta.bcn.cat`) returns `"found": false` for nearly all sensor last-observation queries despite advertising hundreds of CESVA sensors. Only a small subset of sensors appear to have ever transmitted live data, and even those had gaps of weeks. The vertical was dropped in favour of UV and pollen.

S3/Glue/Athena data lake. The proposal allocated 10 hours to an S3-based historical data lake with AWS Glue crawlers and Athena SQL queries for long-range trend analysis. In practice, DynamoDB with a 30-day TTL and sort-key range queries was sufficient for all historical access patterns demonstrated during the project, so this layer was not implemented. The tradeoff is that analytics spanning more than 30 days or requiring SQL aggregations are not possible without exporting from DynamoDB first.

Amazon QuickSight dashboard. Listed as optional in the revised plan, the QuickSight dashboard was not built. The MCP server and Bedrock chatbot cover the primary use cases. QuickSight was identified to be the wrong tool for our plans and ambitions with the project and therefore was dropped.

Bedrock Agents (managed) vs. manual agentic loop. The proposal discussed using the AWS Bedrock Agents service (with its managed orchestration loop and action groups). The final implementation instead runs the tool-use loop manually in the chat Lambda: the Lambda calls `bedrock-runtime.invoke_model` in a loop, handles `stop_reason = "tool_use"` itself, and calls the MCP server for each tool call. This achieves the same result as Bedrock Agents with more transparency and no dependency on the Agents service. Dont understand

3 Architecture and Code Description

3.1 DATASOURCES

List all the datasources that were used and reference them

3.2 Layered Architecture

The system has three functional layers:

1. **Data ingestion layer:** Lambda functions collect live data from external APIs on EventBridge schedules and write append-only records to DynamoDB.
2. **Data serving layer:** The MCP server Lambda reads from DynamoDB and from a small number of live APIs, and responds to JSON-RPC 2.0 tool calls over HTTPS.
3. **AI interface layer:** Claude Haiku (via AWS Bedrock) runs an agentic tool-use loop, calling the MCP server for each data lookup and synthesising a natural-language answer for the user.

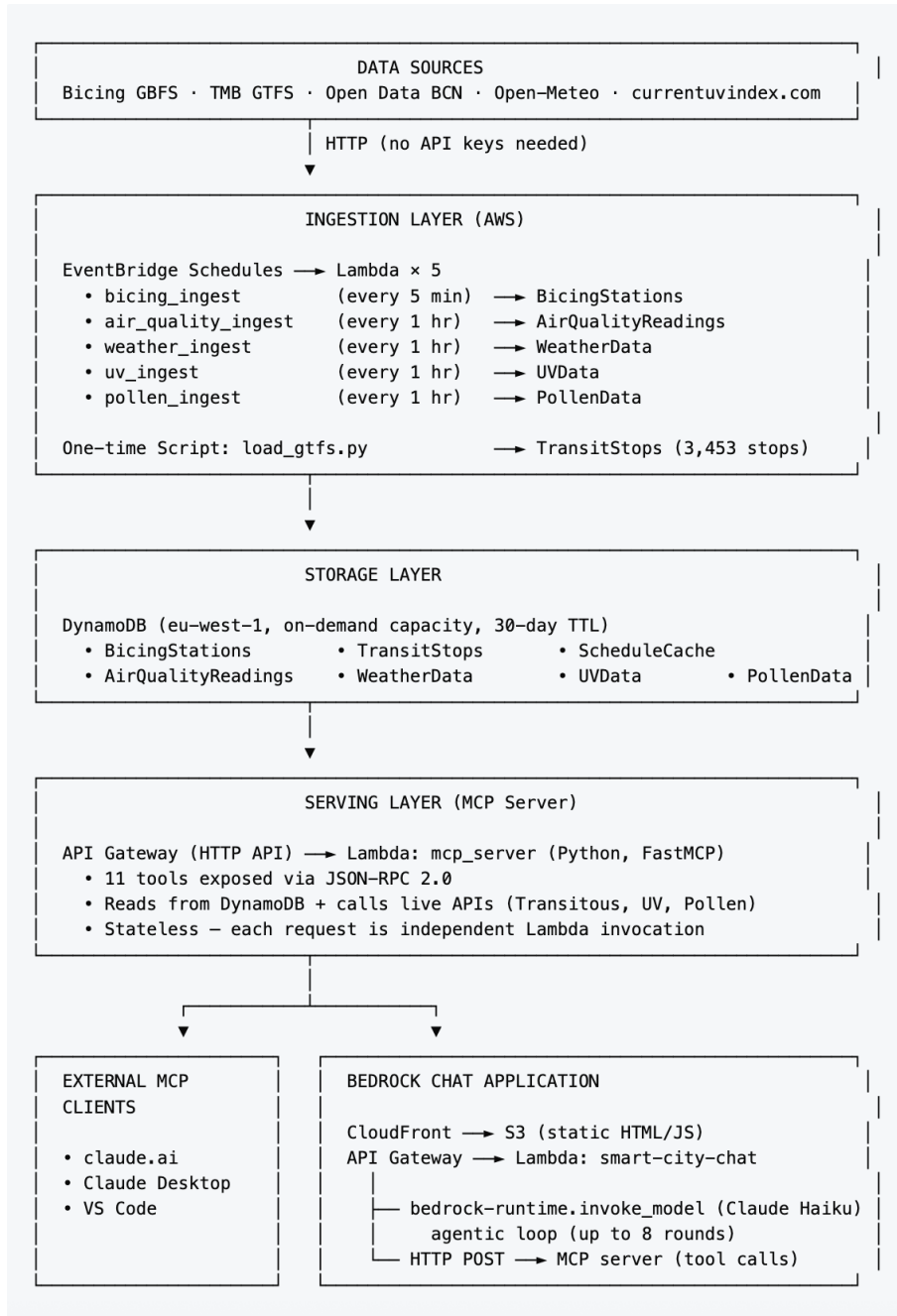


Figure 1: System Architecture

3.3 Data Ingestion Lambdas

Each vertical has a dedicated, single-purpose Lambda function that fetches from one source and writes to one DynamoDB table.

bicing_ingest (every 5 minutes): Calls the citybik.es GBFS mirror (api.citybik.es/v2/networks/bicing) to retrieve live status for all 543 Bicing stations. Each snapshot is written as a new DynamoDB item with a Unix timestamp sort key and a 30-day TTL, enabling historical availability queries.

air_quality_ingest (every hour): Calls the Open Data BCN CKAN datastore API for today's air quality rows from the XVPCA network. The source data format is one row

per (station, pollutant, calendar-day) with hourly columns H01–H24 and validation flags V01–V24. The Lambda scans backward from the current hour to find the most recent validated reading, then writes it with composite primary key `station_pollutant` (e.g. 43_N02) and sort key `hour_ts` (YYYYMMDDHH).

weather_ingest (every hour): Calls the Open-Meteo API for Barcelona city centre. Returns temperature, feels-like temperature, humidity, wind speed and direction, precipitation, cloud cover, and a WMO weather code. Stored in `WeatherData` with fixed PK `barcelona_center` and Unix timestamp SK.

uv_ingest (every hour): Calls `currentuvindex.com/api/v1/uvi` for Barcelona coordinates and stores the UV index in `UVData` with PK `barcelona_center` and SK `hour_ts` (YYYYMMDDHH).

pollen_ingest (every hour): Calls the Open-Meteo CAMS air quality API for the current-hour pollen concentration of six allergen species. Writes one record per species per hour in `PollenData` with composite PK `barcelona_<species>`.

Transit (static load): TMB GTFS data is loaded once via `aws/scripts/load_gtfs.py`, which parses all GTFS feed files, joins routes to stops, and batch-writes 3,453 stop records to `TransitStops` with coordinates, route names, and a `lat_bucket` spatial index attribute.

3.4 DynamoDB Schema Design

Seven DynamoDB tables are used, all on on-demand (pay-per-request) billing mode with 30-day TTL.

BicingStations: Primary key PK = `station_id` (String), Secondary key SK = `updated_at` (Number, Unix epoch). A GSI (`LatIndex`) with PK = `lat_bucket` and SK = `lon` enables a spatial bounding-box pre-filter before the Haversine calculation in the Lambda: query all stations in a given 0.01° latitude bucket (approximately 1 km band), filter by longitude range, then rank by exact distance.

TransitStops: PK = `stop_id` (String), SK = `feed_ver` (String). The feed version as sort key allows safe blue/green GTFS feed transitions: load the new feed, verify it, then let the old version expire by TTL. A GSI (`LatBucketIndex`) uses the same spatial index pattern as `BicingStations`.

AirQualityReadings: PK = `station_pollutant` (String, e.g. 43_N02), SK = `hour_ts` (String, YYYYMMDDHH). The composite PK encodes both station and pollutant so that any time-range query for a specific station/pollutant combination is a simple key query with no scan.

WeatherData, UVData, PollenData: each uses a fixed location identifier as PK and a time-based SK, following the same append-only pattern as `AirQualityReadings`.

The append-only design (never update, always insert with TTL) simplifies concurrency and avoids write conflicts. The 30-day TTL means historical queries have up to 30 days of data once the ingest Lambdas have been running; DynamoDB's TTL deletion is automatic and incurs no additional cost.

3.5 MCP Server (mcp_server.py)

The MCP server is the architectural centrepiece of the platform. It is implemented in Python using the **FastMCP** library for tool registration and schema generation, deployed as an AWS Lambda with an API Gateway HTTP API proxy integration.

The server implements the Model Context Protocol over JSON-RPC 2.0. A client connects in three steps: an `initialize` handshake, a `tools/list` call to discover available tools, and one or more `tools/call` invocations to execute them.

Each tool is a decorated Python function:

Listing 1: Example MCP tool registration with FastMCP

```
@mcp.tool(annotations=_READ_ONLY)
def get_air_quality(lat: float, lon: float, max_stations: int =
    2) -> dict:
    """
    Returns latest air quality readings (NO2, PM10, O3, CO) from
    the nearest
    Barcelona XVPCA monitoring stations. Data live from AWS
    DynamoDB, updated hourly.
    ...
    """
    ranked = sorted(
        _AQ_STATIONS.items(),
        key=lambda kv: _haversine(
            lat,
            lon,
            kv[1]["lat"],
            kv[1]["lon"]
        )
    )[:max_stations]

    # ... DynamoDB query ...
```

FastMCP infers the JSON Schema for each tool from the Python type annotations, avoiding the need to write schemas by hand.

Tool descriptions are carefully engineered. Each description ends with a **PRESENTATION:** instruction block that guides the AI on how to render results (metric cards, charts, follow-up suggestion buttons). This ensures consistent rich output without requiring prompt engineering from the user.

The Lambda handler is fully stateless: each JSON-RPC request is an independent invocation with no shared mutable state. This is critical for concurrent use.

3.6 Demo Web Application (demo/app.py)

The local demo app is a FastAPI application serving a single-page HTML interface (Leaflet.js map) with three tabs:

Route Planner: The user clicks two points on the map; the app calls `get_transit_route` via the Transitous router and draws colour-coded polylines (metro in red, bus in blue, walking in grey). The Transitous API returns encoded polylines (precision 6); these are decoded server-side using the `polyline` library before being sent to Leaflet.

AI Assistant: A chat interface with streaming responses backed by AWS Bedrock. The `/api/chat` endpoint streams newline-delimited JSON events (`text`, `tool_call`, `tool_result`) so the browser can render tool usage in real time as the model thinks. The assistant has five tools: `get_transit_route`, `get_bicing`, `get_transit_nearby`, `get_air_quality`, and a UI-only `place_pins` tool. The `place_pins` tool does not call any external API; it signals the frontend to render coloured map pins for the locations the AI discusses.

City Layers: Fetches a snapshot of live DynamoDB data (all nine air quality stations, up to 300 metro stops, 150 bus stops) and renders them as toggleable map overlays with colour coding by pollution level.

3.7 Publicly Deployed Bedrock Chat Application

The public chatbot is a Lambda function (`aws/lambda/chat/lambda_function.py`) that implements the full Bedrock agentic loop:

1. On cold start, fetch the tool list from the MCP server (cached in Lambda memory for the lifetime of the warm instance).
2. Receive the user message and conversation history from the frontend.
3. Call `bedrock-runtime.invoke_model` with the message history and tool schemas.
4. If `stop_reason == "tool_use"`: for each tool-use block, post a JSON-RPC request to the MCP server, collect the result, append both the assistant turn and tool results to the message history, and repeat from step 3.
5. When `stop_reason == "end_turn"`: return the final text response and the full tool call log to the frontend.

The loop runs for a maximum of 8 rounds. In practice, a composite query such as “is it a good day to cycle?” triggers four tool calls (weather, UV, air quality, pollen) in a single round.

The static frontend is a single HTML file (no build step, no framework) hosted on S3 behind CloudFront. Each assistant response is rendered with a collapsible agent activity block showing every tool call, its input, and its result as syntax-highlighted JSON. The DOM is built with `createElement/appendChild` exclusively to avoid losing event listeners on re-render.

3.8 AWS Resources

Table 2 lists all AWS resources deployed for the project.

All resources are in region `eu-west-1` except CloudFront (global). [Cloudwatch?](#)

Table 2: AWS resources deployed for the project

Resource	Name	Purpose
DynamoDB ($\times 7$)	BicingStations, TransitStops, ...	City data storage
Lambda ($\times 5$ ingest)	smart-city-*-ingest	Data collectors
Lambda ($\times 1$ MCP)	smart-city-mcp	MCP server handler
Lambda ($\times 1$ chat)	smart-city-chat	Bedrock agentic loop
API Gateway ($\times 2$)	MCP + chat endpoints	HTTPS routing
EventBridge ($\times 6$)	*-ingest-schedule	Trigger schedules
S3	barcelona-smart-city-site-*	Static frontend
CloudFront	E2LPOSVMFTXZ34	CDN + HTTPS
AgentCore Gateway	barcelona-smart-city-xwyxovrhex	Managed MCP proxy
IAM roles	smart-city-lambda-*-role	Least-privilege per Lambda

4 Twelve-Factor Methodology

The twelve-factor methodology was used as a design reference throughout the project. Below is a per-factor assessment.

Factor 1 — Codebase

A single GitHub repository (github.com/KubaDusza/barcelona-smart-city) contains all project code: Lambda functions, the MCP server, the demo app, deployment scripts, IAM policy files, and documentation.

Factor 2 — Dependencies

A `requirements.txt` file declares all Python dependencies with pinned versions for the demo app. Lambda deployment bundles include all required packages as a zip archive so the exact same dependency tree is used in every environment. The Lambda Python 3.12 runtime provides `boto3` pre-installed; it is not duplicated in the bundle.

Factor 3 — Config

All configuration that varies between environments is provided via environment variables: `TABLE_NAME`, `AWS_REGION`, `DYNAMO_REGION`, `BEDROCK_REGION`, `BEDROCK_MODEL_ID`, `PORT`. No credentials appear in source code.

Factor 4 — Backing Services

DynamoDB, AWS Bedrock, and external APIs are all treated as attached resources addressed by configuration. The DynamoDB region and table names are environment variables, which would make switching to a different AWS re-setting two env vars. A minor gap: a handful of external API base URLs (`CITYBIKES_URL`, `CKAN_BASE`, `TMB_BASE`) are module-level constants rather than environment variables, making them harder to override without code changes.

Factor 5 — Build, Release, Run

The `aws/deploy.sh` script separates the three stages: *build* (zip-packaging each Lambda with its dependencies), *release* (uploading to S3, updating the Lambda function code), *run* (EventBridge schedules invoke the Lambda on its configured cadence). The `setup.sh` script handles one-time infrastructure provisioning separately from code deployments.

Factor 6 — Processes

All Lambda functions are stateless. The only shared state is DynamoDB (a backing service, as per Factor 4). The `_tools_cache` in the chat Lambda is an in-memory optimisation that survives warm invocations but is safe to lose: on cold start the Lambda refetches from the MCP server. No Lambda writes to a local filesystem.

Factor 7 — Port Binding

The MCP server and demo app export their HTTP service via port binding. Locally: uvicorn on port 8000 (MCP server) or port 8765 (demo app). In production: API Gateway routes HTTPS requests to the Lambda handler, which is the serverless equivalent of port binding.

Factor 8 — Concurrency

AWS Lambda provides automatic horizontal scaling. Each ingest Lambda and each MCP server invocation runs as an independent process. Multiple concurrent users of the MCP endpoint or the Bedrock chatbot trigger independent Lambda instances with no shared mutable state between them.

Factor 9 — Disposability

Lambda functions are inherently disposable. Cold start for these functions is fast (under 3 seconds; warm restart under 100 ms). The stateless design means a failed invocation leaves no side effects: the next invocation simply re-reads from external APIs and re-writes to DynamoDB. EventBridge retries failed Lambda invocations automatically (up to twice by default).

Factor 10 — Dev/Prod Parity

The goal was minimal divergence. The same Python code runs locally (uvicorn) and on Lambda (the `handler` function). The same DynamoDB tables are used from both environments. The main gap is the runtime itself: local execution uses the developer's system Python, while Lambda uses a specific Python 3.12 build. Out of time constraints, scope of the project and effort to set it up we decided against a formal staging environment, like dev, separate from production.

Factor 11 — Logs

All Lambda functions use `print()` statements at key points in their execution path. AWS Lambda automatically captures stdout to Amazon CloudWatch Logs. Every ingest invoca-

tion logs the number of records fetched, written, and skipped, making it easy to detect silent ingestion failures. The development workflow used `aws logs tail /aws/lambda/<name> -follow` to monitor ingest Lambdas in real time.

Factor 12 — Admin Processes

One-off administrative tasks are standalone scripts that run in the same codebase as the application code and connect to the same DynamoDB tables via the same environment variable mechanism: `aws/scripts/load_gtfs.py` for the one-time GTFS load, `aws/scripts/verify_data.py` to confirm all tables are populated. **Fully satisfied.**

Summary

The project substantially follows the twelve-factor methodology. Factors 1, 2, 3, 5, 6, 7, 8, 9, 11, and 12 are fully or substantially satisfied. The main gaps are Factor 4 (a handful of hardcoded external API URLs) and Factor 10 (no formal staging environment separate from production). Both gaps are acknowledged and would be straightforward to close with additional time.

5 Development Methodology

5.1 Team Organisation and Ownership Model

The team of four used a **vertical ownership** model: each person owned a data domain from data source research through to the deployed MCP tool. Horizontal tasks — MCP server integration, deployment infrastructure, documentation — were owned by specific individuals.

- **Jakub Dusza** — Technical lead. Designed and built all deployment infrastructure (`setup.sh`, `deploy.sh`, `pause.sh`, `teardown.sh`); Bicing and Transit vertical (ingest Lambda, GTFS loader, 4 MCP tools); MCP server (`mcp_server.py`, all 11 tools, FastMCP integration, Lambda handler); demo web application (FastAPI backend, Leaflet map, Bedrock chat integration).
- **Mark Atzberger** — Air quality vertical (XVPCA API research, `air_quality_ingest` Lambda, `AirQualityReadings` DynamoDB schema, 2 MCP tools with WHO threshold labelling); AWS architecture design (DynamoDB schema patterns, IAM role structure, EventBridge setup); MCP research for the mid-project presentation; contributions to Bedrock chat application.
- **Jia Lyu** — Weather vertical (`weather_ingest` Lambda, `WeatherData` DynamoDB table, `get_weather` MCP tool); research presentation coordination (structured agenda, distributed sections to teammates, ensured coherence); testing.
- **José Arias Perez** — UV vertical (`uv_ingest` Lambda, `UVData` table, `get_uv_index` with Diffey formula and `get_uv_history` tools); Pollen vertical (`pollen_ingest` Lambda, `PollenData` table, `get_pollen` + `get_pollen_history` tools, 6 species); Bedrock research for mid-project presentation; Bedrock chat application deployment.

5.2 Communication and Shared Documentation

Day-to-day coordination happened over a shared WhatsApp group. Technical handoffs were managed through the GitHub repository. A set of shared Markdown documents served as the authoritative reference for what each person needed to build:

- `PLAN.md`: high-level architecture, task breakdown, and timeline.
- `TEAM_BRIEFING.md`: current state of the system, codebase map, and who did what.
- `docs/MARK_AIR_QUALITY.md`, `docs/JIA_WEATHER.md`, `docs/JOSE_NOISE.md`: vertical-specific guides with data source URLs, DynamoDB schemas, and step-by-step instructions.
- `dynamodb_schema.md`: canonical DynamoDB schema reference for all tables.

This documentation-first approach meant that Jakub could write the deployment scripts and MCP server shell before individual Lambdas were built, and each team member could deploy their Lambda independently by following their vertical guide.

5.3 Working Environments

- **Development**: Local Python 3.9+ virtual environment (venv). Local runs of the demo app connect to live AWS DynamoDB using shared AWS credentials, ensuring the development environment uses the same data as production.
- **Production**: Single shared AWS account (539592518821, eu-west-1). No separate staging environment.
- **Tools**: Git and GitHub for version control; AWS CLI for all infrastructure operations; JetBrains PyCharm for development; CloudWatch console and `aws logs tail` for Lambda debugging.

6 Problems Encountered and Solutions

6.1 Dead Data Sources

Two planned verticals turned out to have no live data despite having documented APIs.

The Sentilo IoT platform (`connecta.bcn.cat`) is Barcelona's open sensor network, listing hundreds of CESVA acoustic noise sensors. After building and testing the Lambda, last-observation queries returned `"found": false` for all but a small subset of sensors, and even those had data gaps of weeks. The API returns HTTP 200 with an empty result rather than an error, making the failure silent.

The CKAN beach water quality dataset had a resource ID that returned HTTP 404.

Solution: Both verticals were abandoned. A 15-minute validation script that performs a live API call and checks for actual data would have caught these issues before any Lambda was written.

6.2 Lambda Function URL Account-Level Block

The first attempt at a public chat API endpoint used a Lambda Function URL (auth type NONE). Despite a correct resource-based policy, all requests returned HTTP 403. Investigation pointed to an account-level SCP (Service Control Policy) or permission boundary that blocks Lambda Function URLs for this account.

Solution: Replaced the Function URL with an API Gateway HTTP API using Lambda Proxy (payload format 2.0) integration. This resolved the 403 immediately, added clean path routing, and gave proper CORS header management.

6.3 IAM 10 Managed Policy Limit

The `smart_city` IAM user accumulated nine managed policies from earlier setup iterations. Adding Bedrock permissions would have exceeded AWS’s hard limit of ten attached managed policies per IAM entity.

Solution: Detached all nine managed policies and replaced them with a single inline policy covering all required services: DynamoDB, S3, Lambda, EventBridge, API Gateway, CloudWatch Logs, IAM (limited), CloudFront, and Bedrock. Inline policies do not count toward the managed-policy attachment limit.

6.4 UV Index Nighttime Edge Case

The `get_uv_index` tool originally identified “today” using the current UTC calendar date (`time.gmtime().tm_mday`). At night (UTC date rolled past midnight), there were no forecast entries for that date yet — the CAMS model’s next-day data starts from the next sunrise. The tool incorrectly returned the highest UV value in the 5-day forecast window instead of today’s expected peak.

Solution: “Today” is now derived from the first non-zero UV entry in the forecast data, which is always the date of the next sunrise regardless of the current UTC time. This fix is correct for both daytime and nighttime calls.

6.5 innerHTML Destroys Event Listeners

In the demo app’s streaming chat interface, tool-call blocks were accumulated by appending HTML strings: `div.innerHTML += newHtml`. The expandable blocks rendered correctly but click handlers had no effect. When `innerHTML` is assigned, the browser serialises the current DOM to HTML and re-parses it, discarding all `addEventListener` bindings on existing elements.

Solution: Replaced all HTML string accumulation with `createElement/appendChild` DOM construction. All click listeners are attached to freshly created elements and are never lost on re-render.

6.6 Pollen History Bootstrap Problem

The Open-Meteo pollen API is forecast-only — it has no endpoint for historical readings. The `PollenData` table can only accumulate data from the Lambda’s deployment date; there is no way to backfill months of historical data.

Solution: This limitation is documented in the tool description. `get_pollen_history` returns whatever has accumulated since deployment, with a note that data starts from that date. A proper fix would use the Open-Meteo historical archive API (if available for pollen) or a third-party pollen database for backfill.

6.7 Bicing BSM API Block

The official BSM Bicing API (`api.bsmsa.eu`) began returning HTTP 503 error 700700 (“API blocked temporarily”) in May 2026, breaking the original ingest approach.

Solution: Switched to the public GBFS mirror via `api.citybik.es/v2/networks/bicing`, which provides the same station data without authentication and has been reliable throughout the project.

7 Services and Resources Used

7.1 AWS Lambda

How the project benefits: Lambda provides zero-infrastructure-management compute. There are no servers to provision, patch, or scale. Each function costs nothing when not invoked (ingest Lambdas are idle most of the time) and scales horizontally to handle concurrent MCP requests automatically. Cold starts are fast for these small Python functions (under 3 seconds; warm under 100 ms).

Alternatives considered:

- *EC2 instance:* full control, but continuous cost, patching, and manual scaling.

7.2 Amazon DynamoDB

How the project benefits: DynamoDB’s single-digit-millisecond read latency is essential for the MCP server, which sits in a live AI conversation where users expect responses in under two seconds. The schema-less document model allowed iterating quickly on table designs without migration scripts. On-demand billing means costs scale exactly with query volume. The 30-day TTL feature automates data expiry without any cleanup logic.

7.3 Amazon EventBridge Scheduler

How the project benefits: Serverless cron with no always-on process. EventBridge handles scheduling reliably with at-least-once delivery. Cost is negligible (first million invocations per month are free).

Alternatives considered:

- *EC2-based cron:* would require maintaining a long-running VM just for scheduling.

7.4 Amazon API Gateway (HTTP API)

How the project benefits: Provides an HTTPS endpoint with TLS certificate management, CORS configuration, and request routing, without managing a load balancer or web

server. The HTTP API (v2) has lower latency and lower cost than the legacy REST API (v1).

Alternatives considered:

- *Application Load Balancer + Lambda*: more flexible routing rules; marginally lower per-request latency at high volume. More configuration overhead at our scale.
- *Lambda Function URLs*: simpler (no API Gateway), but blocked by an account-level policy in our environment (Section 6.2).

7.5 AWS Bedrock

How the project benefits: Managed foundation model API. No model hosting, no billing account with Anthropic, no API key stored as a Lambda secret. The Lambda IAM role grants `bedrock:InvokeModel` permission, meaning no credentials need to be rotated. The EU inference profile (`eu.anthropic.claude-haiku-4-5-20251001-v1:0`) routes requests to EU data centres, supporting GDPR data residency requirements.

Alternatives considered:

- *Anthropic API (direct)*: requires an API key stored as a Lambda environment variable/secret. No meaningful technical difference at our scale, but adds a billing account outside AWS.
- *OpenAI API*: similar managed offering, outside the AWS ecosystem.

7.6 Amazon S3 + CloudFront

How the project benefits: Near-zero-cost static hosting for the web frontend. CloudFront provides HTTPS, CDN caching for fast global delivery, and a stable public URL independent of the Lambda endpoint. Redeployment is a single `aws s3 cp` command followed by a cache invalidation.

Alternatives considered:

- *EC2/Nginx*: adds an always-on instance with operational overhead for a static file.

7.7 AWS Bedrock AgentCore Gateway

How the project benefits: A managed MCP proxy that auto-discovers tools from the MCP server via `tools/list` and exposes them at a signed HTTPS endpoint. Setup required three CLI commands. Although the final chat Lambda calls the MCP server directly (simpler, no SigV4 required since the MCP endpoint is unauthenticated), the gateway is available for future Bedrock Agents or Strands SDK integrations.

Alternatives considered: Writing a custom Lambda proxy to handle tool routing — essentially reimplementing what AgentCore Gateway provides as a managed service.

7.8 MCP (Model Context Protocol)

How the project benefits: MCP is an open standard (originally developed by Anthropic, now donated to the Linux Foundation) that decouples the data layer from the AI client.

By implementing the protocol, the team built the data platform once and can serve any compatible AI tool (Claude Desktop, claude.ai, VS Code Copilot, Cursor, etc.) without custom integration per client. The server is model-agnostic: connecting a different AI client requires only changing the client’s connector URL.

Alternatives considered:

- *REST API*: a standard REST API would require a custom integration for every AI client.
- *OpenAI function calling schema*: narrower standard, not adopted outside the OpenAI ecosystem.

7.9 Open Data Sources

Open Data BCN (CKAN/XVPCA): Barcelona city’s open data portal. The XVPCA air quality dataset is free, hourly-updated, and well-documented.

Open-Meteo: Free weather and air quality (CAMS pollen) API, no key required, sub-second response times, high reliability.

citybik.es: Public GBFS mirror providing Bicing station data. Used after the official BSM API was blocked.

currentuvindex.com: Free UV index API backed by the Copernicus CAMS model.

Transitous: Open-source European intermodal router. Provides realistic multi-leg journey planning across Barcelona’s metro and bus network.

TMB (Transports Metropolitans de Barcelona) API: Used in the demo app for live bus arrivals. Requires a free API key from `developer.tmb.cat`.

8 Hours Invested

The original estimate from the first draft was **150 hours total** (approximately 37.5 hours per person). Table 3 shows the actual hours invested per person per task category, estimated by each team member and compared against the original plan.

Table 3: Hours invested per team member and task (estimated)

Task	Jakub	Mark	Jia	José	Total
Project management & coordination	8	2	3	2	15
MCP onboarding & research	3	3	3	3	12
AWS setup, IAM, infra scripts	10	3	1	1	15
Bicing + Transit vertical	15	—	—	—	15
Air quality vertical	—	12	—	—	12
Weather vertical	—	—	10	—	10
UV vertical	—	—	—	8	8
Pollen vertical	—	—	—	8	8
Noise vertical (abandoned)	—	—	—	5	5
MCP server (<code>mcp_server.py</code>)	12	2	—	1	15

Task	Jakub	Mark	Jia	José	Total
Demo web application	10	2	2	1	15
Bedrock chat application	3	5	—	10	18
Testing & debugging	6	4	4	5	19
Presentation (mid-project + final)	3	3	4	3	13
Report writing	2	5	3	3	13
Total actual	72	41	30	50	193
Planned (per person)	37.5	37.5	37.5	37.5	150
Deviation	+34.5	+3.5	−7.5	+12.5	+43

8.1 Analysis of the Deviation

The project ran approximately **43 hours over the original estimate** (29% overrun). The primary causes:

1. **Dead data sources (noise vertical, ≈12 hours):** Researching, building, deploying, and then abandoning the Sentilo noise vertical consumed roughly 12 hours across José. Additionally, diagnosing why the CKAN beach water quality resource ID 404'd cost further time.
2. **Infrastructure debugging (≈12 hours on Jakub):** The Lambda Function URL account-level block, the IAM 10-managed-policy limit, and debugging EventBridge role propagation each required investigation and workarounds that were not anticipated in the original estimate.
3. **Bedrock chat deployment complexity (≈10 hours across Jakub and José):** Deploying a production-quality web app (S3 + CloudFront + API Gateway + IAM policy consolidation) involved more resources and debugging than anticipated.
4. **Scope expansion with UV and pollen (≈8 hours on José):** These two verticals were not in the original plan; their development time was not budgeted.

Jia's under-run reflects that the weather vertical used a straightforward, well-documented data source (Open-Meteo) and had a narrow scope.

8.2 How the Deviation Could Have Been Reduced

- **Validate data sources before building:** A 15-minute script that calls the API and checks for live data at the start of each vertical would have saved the 12 hours spent on the noise vertical.
- **Infrastructure-as-code from the start:** Codifying all AWS resources in a template (e.g. AWS SAM or CloudFormation) earlier would have exposed IAM limits and permission gaps before they caused debugging sessions.
- **Staging environment:** A separate staging AWS account or Lambda alias would have allowed Bedrock and API Gateway configurations to be tested before touching the shared production environment.

- **Plan scope expansion explicitly:** UV and pollen were good additions, but their hours should have been budgeted when the decision was made to include them, rather than discovered retrospectively.

9 Conclusion

The Barcelona Smart City Insights Platform successfully delivers a live, multi-domain city data API accessible to any MCP-compatible AI assistant and through a self-contained public Bedrock chatbot. By building a standard protocol interface over AWS serverless infrastructure, the project demonstrates that Barcelona’s open data — valuable but practically inaccessible to most tools and citizens — can be made conversationally accessible without a custom app, without API keys for end users, and without any change in how people use the AI tools they already have.

The system covers six data domains (Bicing bike-share, TMB transit, air quality, weather, UV, pollen), exposes 11 MCP tools, and runs two AI interfaces on top of a shared data layer — showing that the investment in the data infrastructure multiplies across every interface built on it.

Key technical lessons learned:

1. Always validate external data sources with a live test request before building infrastructure around them.
2. DynamoDB’s pay-per-request model, sub-millisecond latency, and automatic TTL make it a natural fit for real-time city data at small to medium scale. The schema-less model supports rapid iteration without migration overhead.
3. The twelve-factor methodology aligns naturally with Lambda-first architecture; the main gap is dev/prod parity (no staging environment), which would require additional time to address properly.
4. Tool description engineering is non-trivial: poorly described tools lead to poor AI orchestration. The investment in clear, specific tool descriptions — including the **PRESENTATION**: blocks and Haversine-based coordinate hints — pays dividends in user experience.
5. AWS IAM managed-policy limits are a real operational concern when iterating across many services; a single comprehensive inline policy (or role-per-service) is more maintainable than accumulating managed policies.